Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 (MCT3)

Zielsetzung

Sie vertiefen Vorlesung 2: Lokalitätsprinzip, AMAT, Cache-Mapping (Direct Mapped / Set-Associative), Schreibstrategien und cache-freundlichen C-Code (Row-Major vs. Column-Major).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 14 P, Aufgabe 2: 8 P (Gesamt: 22 P)

Teil 1: Theorie — Speicherhierarchie und Caches

Aufgabe 1.1: Das Lokalitätsprinzip

Caches funktionieren nur, weil Software kein völlig zufälliges Zugriffsverhalten zeigt.

1. Erklären Sie den Unterschied zwischen zeitlicher Lokalität (Temporal Locality) und räumlicher Lokalität (Spatial Locality).
2. Warum sind Arrays aus Sicht des Caches typischerweise effizienter zu verarbeiten als verkettete Listen (Linked Lists)?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.2: AMAT-Berechnung

Die Average Memory Access Time (AMAT) ist eine zentrale Kenngröße der Speicherarchitektur.

Ein Prozessor habe:

- Hit Time des L1-Caches: 1 Takt
- Miss Penalty (Zugriff Hauptspeicher/DRAM): 100 Takte
- Hit Rate des L1 bei einem Programm: 95 %

1. Berechnen Sie die AMAT. Geben Sie Formel und Rechenweg an.
2. Welche Auswirkungen hat es auf die AMAT, wenn Pointer-Ketten die Hit Rate auf 50 % senken?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.3: Cache-Mapping und Adresszerlegung

Eine 32-Bit-Adresse wird in Tag, Index und Offset zerlegt.

1. Welche Aufgabe haben Tag, Index und Offset beim Lookup im Cache?
2. Erklären Sie Thrashing bei einem Direct-Mapped-Cache anhand zweier Arrays A und B.
3. Wie mildert ein N-Way Set-Associative Cache Thrashing? Warum sinkt dabei typischerweise die Zahl der Index-Bits?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.4: Schreibstrategien

Beim Schreiben droht Inkonsistenz zwischen L1-Cache und RAM.

1. Vergleichen Sie Write-Through und Write-Back hinsichtlich Bus-Belastung und Performance.
2. Welche Rolle hat das Dirty-Bit bei Write-Back, und wann werden Daten in den RAM geschrieben (Eviction)?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Teil 2: Praxis — Cache-freundlicher C-Code

Aufgabe 2.1: Row-Major vs. Column-Major

Gegeben sind zwei Snippets, die ein 1000×1000-Integer-Array auf 0 setzen. C speichert 2D-Arrays **zeilenweise** (Row-Major).

Variante A:

```
for (int row = 0; row < 1000; row++) {
    for (int col = 0; col < 1000; col++) {
        bild[row][col] = 0;
    }
}
```

Variante B:

```
for (int col = 0; col < 1000; col++) {
    for (int row = 0; row < 1000; row++) {
        bild[row][col] = 0;
    }
}
```

1. Welche Variante ist cache-freundlich, welche problematisch? Begründen Sie physikalisch (Lokalität, Cache-Lines).
2. Was passiert in Variante B bei jedem Schritt der inneren Schleife bezüglich der geladenen Cache-Line?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.2: Messung (optional / Labor)

Übertragen Sie Variante A (und zum Vergleich Variante B) in einen Simulator mit Daten-Cache (z. B. Ripes). Aktivieren Sie den D-Cache und vergleichen Sie die Hit-Rates.

Ihre Lösung

(Notieren Sie Ihre Beobachtungen hier.)